

UNIVERSIDADE DE BRASÍLIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Chat Peer-to-Peer (P2P)

Relatório do Projeto Final de Redes de Computadores

Alunos(grupo 8):

Kaio Santos Araújo - 242009972

Caio Dias Fleury - 242009909

João Paulo Silva Mendes - 242026187

Disciplina: Redes de Computadores

Professor: Marcos Fagundes Caetano

Semestre: 2026/1

Brasília – DF
Junho de 2026

Sumário

1	Introdução	2
2	Requisitos	2
2.1	Requisitos mínimos	2
2.2	Compilação	2
2.3	Bibliotecas necessárias	2
2.4	Instalação das dependências	2
2.5	Passos para executar o programa	3
3	Arquitetura da Solução	3
3.1	Componentes principais	3
3.2	Fluxo de interação entre componentes	3
4	Protocolos Implementados	4
4.1	Rendezvous	4
4.2	HELLO / HELLO_OK	4
4.3	PING / PONG	5
4.4	SEND / ACK	5
4.5	PUB	5
4.6	BYE / BYE_OK	6
5	Funcionamento Geral do Sistema	6
5.1	Inicialização do programa	6
5.2	Registro no Rendezvous	6
5.3	Descoberta de peers	6
5.4	Estabelecimento das conexões TCP	6
5.5	Troca de mensagens	7
5.6	Monitoramento das conexões	7
5.7	Reconexões	7
5.8	Encerramento da aplicação	7
6	Observabilidade e Logs	7
6.1	Monitoramento e registro de eventos	7
6.2	Níveis de log	8
7	Considerações Finais	8

1 Introdução

O projeto analisa um sistema de chat peer-to-peer (P2P) implementado em Python. O objetivo da aplicação é permitir que peers se registrem em um servidor rendezvous central, descubram outros peers ativos em seu namespace e estabeleçam conexões TCP diretas para troca de mensagens.

A solução resolve o problema da descoberta de peers em uma rede distribuída sem exigir que todas as mensagens passem por um servidor central. O servidor rendezvous atua apenas como ponto de encontro para registro e descoberta de peers. A comunicação de usuário a usuário é feita por conexões TCP diretas entre os peers.

A arquitetura P2P adotada combina:

- Um cliente TCP local em cada peer
- Um mecanismo de descoberta via servidor central rendezvous
- Conexões diretas para envio de mensagens e publicação em grupo

É importante notar que o código do projeto contém o cliente de rendezvous e o peer local, mas não inclui a implementação do servidor rendezvous central.

2 Requisitos

2.1 Requisitos mínimos

- Python 3.10 ou superior.
- Sistema operacional compatível com Python e sockets TCP.
- Rede com acesso ao servidor rendezvous configurado em `config.json`.
- Portas:
 - Porta do servidor rendezvous configurada em `host/port` (padrão 8080);
 - Porta do peer local em `peer_port` (padrão 5000);
 - Porta da interface web em `webapp_port` (padrão 8000).

2.2 Compilação

A linguagem é interpretada em Python. Não há etapa de compilação.

2.3 Bibliotecas necessárias

As dependências estão listadas em `requirements.txt`.

2.4 Instalação das dependências

Com ambiente virtual ativado:

```
python -m venv .venv
source .venv/bin/activate # Linux/macOS
.venv\Scripts\activate # Windows
pip install -r requirements.txt
```

2.5 Passos para executar o programa

1. Baixar o arquivo .zip e extrair ou clonar o repositório
2. Criar e ativar ambiente virtual
3. Instalar dependências com `pip install -r requirements.txt`
4. Ajustar `config.json` se necessário
5. Iniciar o peer.

Execução no modo web

```
python main.py
```

Após iniciar, acessar `http://localhost:8000` e informar nome e namespace.

Execução no modo CLI

```
python main.py --cli
```

Nesse modo, o peer usa `name` e `namespace` definidos em `config.json`.

3 Arquitetura da Solução

A arquitetura do projeto é modular e orientada a tarefas assíncronas.

3.1 Componentes principais

Os principais módulos são:

- `main.py`: ponto de entrada e orquestrador de tarefas assíncronas.
- `config.py`: leitura de `config.json` e exportação de constantes.
- `server.py`: servidor TCP local do peer e fila de mensagens.
- `rendezvous.py`: cliente de registro, descoberta e desregistro no servidor rendezvous.
- `router.py`: roteador de mensagens recebidas para handlers específicos.
- `connection.py`: mecanismo de keep-alive, reconexão e encerramento limpo.
- `peer_table.py`: tabela de peers conhecidos e estado de conexão.
- `handlers/`: handlers específicos de protocolo, incluindo `hello`, `ping`, `send` e `bye`.
- `cli.py`: interface de linha de comando e envio de mensagens.
- `app.py`: interface web com Quart e Socket.IO, integração do terminal e chat.

3.2 Fluxo de interação entre componentes

A interação principal é:

1. `main.py` inicializa o servidor web opcional, o servidor TCP local e as tarefas de descoberta, roteamento e keep-alive.
2. `server.py` aceita conexões TCP de outros peers e cria tarefas `peer_listener` para cada socket.
3. Cada mensagem recebida é convertida de JSON e colocada em `message_queue`.
4. `router.py` consome a fila e chama o handler correto de acordo com o campo `type`.

5. `rendezvous.py` faz registro periódico, descoberta periódica e desregistro no servidor rendezvous.
6. `connection.py` monitora conexões já estabelecidas, envia PINGs e aciona reconexão quando necessário.
7. `cli.py` lê comandos do terminal, monta mensagens SEND e PUB e aguarda ACKs.
8. `app.py` expõe o estado de peers e envia eventos de chat para o navegador.

4 Protocolos Implementados

O projeto implementa protocolos de controle e dados em JSON sobre TCP.

4.1 Rendezvous

O cliente envia comandos ao servidor central por conexões temporárias.

REGISTER

- finalidade: registrar o peer no servidor rendezvous.
- fluxo: abrir conexão TCP com `HOST:PORT`, enviar JSON com `type: "REGISTER"` e campos `name`, `namespace`, `port`, `ttl`.
- resposta: servidor retorna JSON com `status "OK"` ou `"ERROR"`.
- momento: ao iniciar e depois em loop periódico em `register_loop`.

UNREGISTER

- finalidade: remover o peer do servidor rendezvous durante encerramento.
- fluxo: abrir conexão, enviar `type: "UNREGISTER"` com `name`, `namespace`, `port`.
- resposta: valida erros específicos como `bad_namespace` ou `peer_not_registered`.
- momento: em `close_all_connections` no encerramento.

DISCOVER

- finalidade: obter lista de peers conhecidos no rendezvous.
- fluxo: abrir conexão temporária e enviar `type: "DISCOVER"`, opcionalmente com `namespace`.
- resposta: lista de peers em `data["peers"]` quando `status` é `"OK"`.
- momento: em loop periódico no `discover_loop`.

4.2 HELLO / HELLO_OK

HELLO

- finalidade: iniciar handshake e estabelecer conexão TCP direta com outro peer.
- campos: `type: "HELLO"`, `peer_id`, `version`, `features`, `ttl`.
- momento: enviado ao tentar conectar-se a um peer descoberto no `cadastrar_peers`.
- fluxo: `handlers/hello.py:hand_shake` abre conexão, cria evento e aguarda `HELLO_OK`.

HELLO_OK

- finalidade: confirmar handshake ingressante.
- campos: `type:"HELLO_OK"`, `peer_id`, `version`, `features`, `ttl`.
- momento: resposta imediata a HELLO no `hello_handler`.
- processamento: libera o evento esperado pelo peer que iniciou o handshake.

4.3 PING / PONG

PING

- finalidade: manter conexões vivas e detectar queda de peers.
- campos: `type:"PING"`, `msg_id`, `timestamp`, `ttl`.
- momento: enviado por `core/connection.py::keep_alive` quando um peer está ocioso por mais de `PING_INTERVAL`.
- fluxo: cria evento assíncrono e aguarda resposta PONG em até 2 segundos.

PONG

- finalidade: confirmar que o peer remoto ainda está acessível.
- campos: `type:"PONG"`, `msg_id`, `timestamp`, `ttl`.
- momento: enviado como resposta a PING no `ping_handler`.
- processamento: evento associado ao `msg_id` é liberado pelo `pong_handler`.

4.4 SEND / ACK

SEND

- finalidade: enviar mensagem direta a um peer específico.
- campos: `type:"SEND"`, `msg_id`, `src`, `dst`, `payload`, `require_ack`, `ttl`.
- momento: enviado pela CLI em `interfaces/cli.py::send`.
- fluxo: grava evento em `open_send` e aguarda ACK por até 5 segundos.

ACK

- finalidade: confirmar entrega de mensagem SEND.
- campos: `type:"ACK"`, `msg_id`, `timestamp`, `ttl`.
- momento: enviado por `send_handler` ao receber SEND.
- processamento: `ack_handler` libera o evento de espera do emissor.

4.5 PUB

- finalidade: publicar mensagem para um grupo de peers.
- campos: `type:"PUB"`, `msg_id`, `src`, `dst`, `payload`, `require_ack`, `ttl`.
- momento: enviado pela CLI em `pub()`.
- processamento: `pub_handler` registra a mensagem e encaminha para a interface web.

4.6 BYE / BYE_OK

BYE

- finalidade: notificar peers sobre desligamento.
- campos: `type:"BYE"`, `msg_id`, `src`, `ttl`.
- momento: enviado em `close_all_connections` durante encerramento.
- fluxo: as conexões são fechadas logo após enviar a mensagem.

BYE_OK

- finalidade: confirmar recepção do BYE.
- campos: `type:"BYE_OK"`, `msg_id`, `src`, `dst`, `ttl`.
- momento: enviado por `bye_handler` ao receber BYE.
- processamento: `bye_ok_handler` apenas registra a confirmação em log.

5 Funcionamento Geral do Sistema

5.1 Inicialização do programa

O arquivo `main.py` inicia:

- servidor web em `WEBAPP_PORT` se o modo padrão for usado
- registro no rendezvous
- servidor TCP local em `PEER_PORT`
- descoberta de peers
- roteador de mensagens
- keep-alive
- loop de CLI

Em modo web, a interface solicita `name` e `namespace`; em modo `-cli`, utiliza valores de `config.json`.

5.2 Registro no Rendezvous

Após obter nome e namespace, `main.py` chama `register_handler`. O registro abre conexão TCP com o servidor rendezvous, envia comando `REGISTER` e espera resposta.

5.3 Descoberta de peers

O peer periodicamente executa `discover_handler` para solicitar `DISCOVER`. A cada 30 segundos o código tenta descobrir peers ativos e registra seus dados em `connected_peers`.

5.4 Estabelecimento das conexões TCP

Para cada peer descoberto, `handlers/hello.py::cadastrar_peers` faz:

1. registrar o peer em `PeerTable`
2. iniciar handshake via `hand_shake`
3. abrir conexão TCP de saída ao peer

4. enviar HELLO
5. aguardar HELLO_OK

O peer receptor responde com HELLO_OK e grava a conexão com status CONNECTED.

5.5 Troca de mensagens

Mensagens P2P são trocadas por:

- SEND + ACK para mensagens diretas
- PUB para publicações em grupo
- PING/PONG para keep-alive
- BYE/BYE_OK para encerramento

O roteador centraliza o processamento das mensagens recebidas e aciona handlers adequados.

5.6 Monitoramento das conexões

O keep_alive executa em loop:

- calcula tempo desde o último ping em cada peer
- envia PINGs quando o peer está ocioso
- marca conexão como TRYING_CONNECTION se o PONG não chega
- inicia reconexão assíncrona

5.7 Reconexões

A rotina try_to_reconnect tenta reestabelecer conexão com peer falho:

- usa hand_shake novamente
- repete até PEER_RECONNECT_TRIES
- aplica backoff exponencial entre tentativas

5.8 Encerramento da aplicação

No comando /quit ou KeyboardInterrupt:

- close_all_connections envia BYE para peers ativos
- encerra sockets TCP
- chama unregister no servidor rendezvous
- cancela tarefas assíncronas restantes

6 Observabilidade e Logs

6.1 Monitoramento e registro de eventos

A aplicação usa logging para registrar eventos em vários componentes:

- MAIN, SERVIDOR, RENDEZVOUS, ROTEADOR, CONEXÃO, HELLO, PING, SEND, BYE, WEBAPP, CLI.

Os logs ajudam a identificar falhas de handshake e timeouts, detectar peers que perderam conexão, verificar status de discovery e registro no rendezvous, acompanhar o fluxo de mensagens recebidas e enviadas.

6.2 Níveis de log

A configuração inicial usa `logging.DEBUG`, o que permite capturar:

- **DEBUG**: mensagens de debug e detalhamento do fluxo
- **INFO**: eventos normais como registro, conexão e mensagens
- **WARNING**: timeouts e situações de reconexão
- **ERROR**: falhas de comunicação, exceções e erros de protocolo

7 Considerações Finais

A implementação expõe uma solução P2P funcional, realizando efetivamente o mecanismo de um chat P2P baseado em rendezvous e conexões TCP. A arquitetura modular facilita entender cada responsabilidade: descoberta, handshake, roteamento, keep-alive e interface.